Style transfer for symbolic drums: A deep learning approach.

By Ali Ncibi, Clement Maurel *Telecom Paris*

(Ondřej Cífka, Umut Şimşekli and Gaël Richard)

ABSTRACT

In this project we aim to explore some deep learning techniques for automatic music generation, namely seq2seq patterns. We will be mainly focusing on the problem of style transformation also known as style transfer. To get started with this task we choose to study and understand Groove2Groove (2), a one-shot learning framework that solves the task of style transfer between symbolic musical accompaniments in a supervised fashion. The system will then be tweaked and evaluated under different schemes and we will be particularly interested in the generation of sequences of drums. Our objective will be to study the generated outputs in terms of adequacy with some user-defined objective measures, and assess the quality of style transfer process with respect to them.

Keywords: Style transfer, Seq2seq, Symbolic music, Drums, Groove pattern, Similarity score

1. Introduction

Style transfer is the process of changing the style of an image, video, audio clip or musical piece (content input) so as to match the style of a given example (style input). An ideally expected output for the task would be for instance a new song in the target style that additionally matches the content song in some sense. In general, Musical style might refer to any of the numerous aspects of music (composition, arrangement, performance or timbre). This leads to the need for a clear definition of style (i.e. what is transferred from Z) and content (what is retained from X) in the style transfer task.

With Groove2Groove platform, we adhere to the previous traditional definition of style transfer and apply it within the domain of symbolic music i.e. music represented as a set of events rather than a waveform. The model, following an encoder-decoder pattern, transforms a source song X so as to match the style of a given target song fragment Z, where both X and Z are provided as MIDI files. Initial Groove2Groove is particularly interested in accompaniment styles in the context of jazz and popular music: the content it tries to retain from X is the chord chart behind that song, and the style is the process of transforming this chord chart to a given accompaniment Z.

We will be particularly interested in the generation of drums sequences i.e when we want to mimic the style of a given target drums sequence. Drums sequences are in reality repetitive and potentially contain repetitive patterns in terms of velocity, timing, duration of played notes. Tracking the extent of implication of such patterns in the transfer process for drums, as well as assessing the quality of the style transfer task are our main objectives during this work.

We start by exploring Groove2Groove with its initial architecture as presented in Section 2. Next we try to adapt the platform (architecture, synthetic database) to our purpose following different tweaking schemes as explained in section 3. Before analyzing the final generated drums, we introduce a set of hand-crafted objective measures that help track some objective patterns (Section 4) and judge the quality of this particular transfer task. Finally, we draw empirical comments about the quality of style transfer task in the sense of the proposed metrics. We assess individually similarity with content and style fit with target (Section 5).

2. Groove2Groove platform

The Groove2groove model is a deep learning model with supervised learning whose task is to transfer style from one instrumental music to another [1]. From a music representing the content (chords, melody, structure of the piece alternating verses, choruses, bridges, solos ...), Groove2Groove applies the style of another music (rhythm, choice of instruments, the "color" of the piece, slight harmonic variations) to form a new instrumental accompaniment. The music considered is "symbolic", i.E represented as a sequence of notes, defined by their position

[1] https://groove2groove.telecom-paris.fr

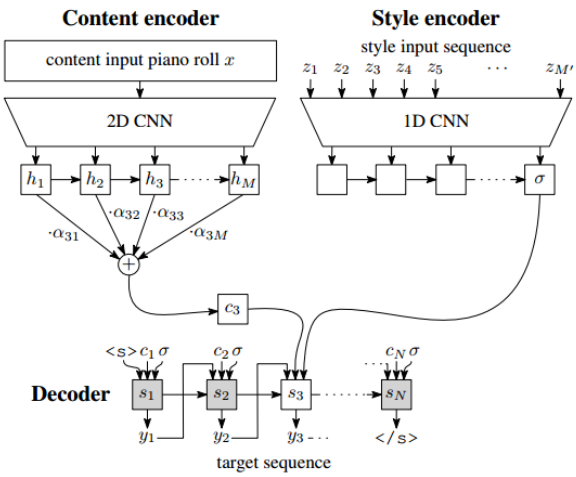

Fig. 1. A detailed view of the initial encoder-decoder architecture used in Groove2Groove, with focus on 3rd decoding step.

in the music, their pitch, their intensity (the force with which a piano key is struck) and their duration.

2.1. Architecture of the model

A neural network following the encoder-decoder pattern, trained to map two input fragments X and Z directly to the output, along with a synthetic data generation scheme to supply it with parallel training examples. The network consists of two encoders and a decoder that process the respective inputs:

- An encoder for content X
- An encoder for style Z
- And a decoder, that subsequently generates the output song.

The content encoder receives the content input X containing all tracks of the song except drums, while the style encoder receives Z^i the ith track of the style input. The decoder then combines the representations computed by the two encoders, and the label y^i, to generate the corresponding ith output track, $\hat{y}^i$. Note that neither the index i nor any other information about the expected output track is given explicitly; instead, the model must infer all the necessary information from Z^i itself.

2.2. Initial encoding decoding scheme

Content input X is encoded as piano roll matrix. It passes through two 2D convolution layers. The flattened 3D features map is then fed to a RNN cell in a many-to-many fashion. An attention mechanism is used to build the input to the decoder out of the content's hidden states sequence. Style input instrument Z^i is encoded using a beat-relative encoding method. It's first projected into some embedding space and the result passes through two 1D convolution layers. The flattened output is fed

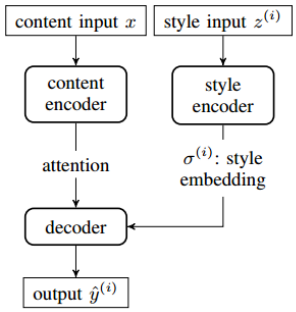

Fig. 2. The model generates the ith track of the output $\hat{y}$ given the content input x and the ith track of the style input z. The overall output $\hat{y}$ is obtained by combining all the $\hat{y}^{(i)\prime}s$

to a RNN cell in a many-to-one fashion. The last hidden state encodes style information from ith instrument, and is fed as 2nd input to the decoder. The decoder receives two encoding, one resulting from content X and the other resulting from style Z (actually instrument Z^i), along with y^i (ground-truth track for instrument i). It generates the output sequence $\hat{y}^i$ subsequently.

2.3. Synthetic data allowing supervised learning

In Groove2Groove a smoothed language model was trained on a corpus of chord charts of over a thousand jazz standards. Using that smoothed LM, new chord charts were sampled. The harmonic variability of jazz, the smoothness of the LM, and this resampling procedure allowed covering diverse styles. Next, Band-in-a-box software was used to render those chord charts into MIDI songs. Band-in-a-box allows rendering a single chord chart into N different styles. Alike, we can build triplets of MIDI songs (A^S, B^T, A^T) such as:

- A^S: Song rendered from chord chart A in style S, content input
- B^T: Song rendered from chord chart B in style $T \neq S$, style input
- A^T: Song rendered from chord chart A, in style T, ground-truth

The synthetic database was created using band-in-a-box, a music accompaniment software package capable of generating musical accompaniments in more than 1500 different musical styles (Pop, Rock, Jazz, Blues, Funk, Country... and many sub-styles) given a chord table. By generating an accompaniment for the same chord table in 2 styles S and T, and an accompaniment for a different chord table in style T, we get respectively the "content" music (chord table 1 and style A), music resulting from the style transfer (the "target", chord table 1 and style B) and the "style" music (chord table 2 and style B). By using 1200 chord tables and about 1500 preset styles of band-in-a-box, cutting the resulting files every

8 bars, and building all possible triplets, we obtain more than one million triplets (content, style, target) used to train the algorithm in a supervised way.

2.4. Notes on style transfer process

When transferring style, the content and the style probably contain different instruments. Since it is not necessarily possible to establish a correspondence between the instrument sets of the content and the style, the instrument sets of the style are kept as the instrument set for output. Thus, we ignore the different instruments of the content, which are all considered to be a single instrument playing all the notes of the content. To generate the output tracks, resulting from the style transfer, each instrument track of the style (guitar, piano, bass, drums etc.) is considered individually, as well as the Piano roll of the content (which contains the notes/melodies/chords of the content, but without distinction of instruments). If the style track contains a guitar track, a piano track and a drum track, the output resulting from the style transfer will also contain a guitar, piano and drum track, independently of the instruments of the content (but based on the notes of the content).

The content encoder does not distinguish between the different content's instruments. While this is almost unimportant for the instruments, which play in tune with each other (it is considered that melodies and chords played by a violin, double bass and piano are played by the same instrument), it poses a representation problem for the drums, whose "notes" are not comparable with those of the other instruments. The drum set is the only instrument that does not play notes (the "pitch", a notation also used for the drum track, does not correspond to the pitch of the note but to the type of percussion used, for example a pitch of 38 for a type of snare drum). We cannot consider that the same instrument plays the drums and a piano or a violin for example. As Piano roll does not allow to distinguish between different instruments, the drum track must be removed from the content. In practice, if the style contains a drum track, Groove2groove is able to generate it by "guessing" its shape from the other instruments in the content, but the results are not satisfactory. Groove2groove "doesn't know" that the track to be generated is a drum track, and since the structure of a drum track is very different from that of the other instruments (the drums don't play notes, the pitch doesn't correspond to different note pitches but to different percussion, the drums are supposed to play very repetitive patterns...), the result seems quite random and not very repetitive when listening to it.

3. Models used for drums generation

Based on the previous platform, we train 3 different models to learn style transfer for drums sequences. Details about the experiments and the modifications to consider for each model

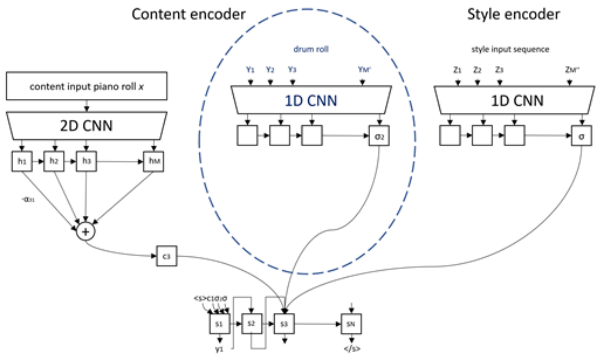

Fig. 3. Adding content's drums encoder to G2G Basic architecture.

are given below.

The first model will be exactly the same one used in Groove2Groove platform, trained similarly on the synthetic dataset. We refer to this model as G2G Basic. We use it to generate drums sequences i.e when the style inputs are drums tracks.

We noticed the absence of drums information from content's input (ignored in the initial platform because drums are not suitable for piano roll encoding used to encode the content) during the encoding-decoding process. We propose a second model with a small architectural modification to handle the absence of this piece of information. It adds a separate encoder for drums track of content song, previously ignored in G2G Basic. Training will be performed on synthetic dataset. We refer to this model as G2G Plus.

And finally, we propose a model that uses the same architecture as in G2G Basic, but trained on a different database (Lakh MIDI dataset) in a self-supervised fashion, preserving the supervision in the initial platform. We refer to this model as G2G Lakh.

3.1. G2G Plus

The piano roll representation of the drums track seems to be less informative. That's why maybe we can't just add them to the piano roll representation of the content song. Our idea is to extract the drums track from the content song separately, encode it, and feed the result to the decoder. For that, we choose to encode the drums sequence of the content input in what we refer to as Beat-Relative encoding (sequence of tokens representing 3 types of events).

That type of encoding was actually used in the initial model to encode the style sequences. The idea is simply to reproduce the encoding scheme for the style input (yellow) and use it

[Tab. 1.]

	train	test	val
MIDI files	124,992	26,784	26,785
MIDI cleaned	57,631	12,451	12,396
Segments	735,803	158,196	158,835

to encode the drums track (purple) extracted from the content song. The obtained latent vector is then fed to the decoder along with other inputs (content input and style input) just like in the original version of Groove2Groove. Note that content input always contains all tracks except drums track, and is always presented in piano roll format.

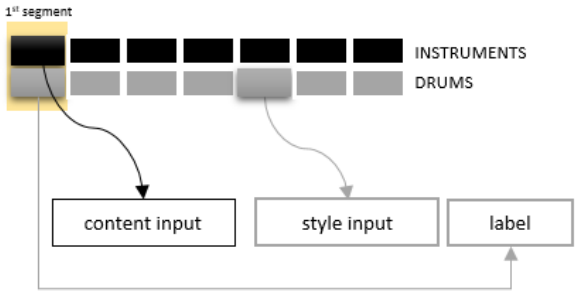

Fig. 4. G2G Lakh training scheme.

3.2. G2G Lakh

The Lakh MIDI dataset [2] contains 176,581 MIDI files gathered from multiple sources which correspond to the MIDI transcription of existing songs. They contain several instruments and more than half of them contain drums. The idea is to create a model that merges the style of a drums track with the content of a song (that doesn't have drums), to infer the drums track for the content.

We reuse the original Groove2groove model but train it differently. Since G2G is a supervised algorithm, we need to build triplets (content, style, target) where target results from the transfer of style percussion from style to content. The Groove2groove model would then be trained to produce only the drum track instead of the instrument tracks as it does initially. To do so, we could use again the synthetic database that is used to train the initial model but we thought it would be interesting to create a new one, composed of existing and very diverse musics. To create a training triplet from a MIDI file containing instrument and drum tracks, we cut it as follows: we extract a segment of the song (8 bars for example, which corresponds to 32 seconds if the song is at 60 beats per minute), we recover the instrument tracks that will make up the content, the drum track will be the target; for the style, we select another segment of the song and keep only the drum track. In this way, the model will learn how to generate a drum track in the style and adapted to the content (since the target is the field reality corresponding to this piece of music). Those triplets are then fed to the G2G model, as represented in figure 4.

The pre-processing of the MIDI files of the dataset consists in: removing corrupted files, keeping only songs that have 4/4 time signature, normalizing the tempo to 60BPM, merging equivalent instruments, keeping only segments that contain drums and other instruments. We split the MIDI files in 3 folders: train for 70% of them, test and validation 15% each.

Tab. 1. shows the number of MIDI files added to each folder, the number of different songs that passed through the pre-processing and the number of triplets after the splitting in 8 bars segments.

[2] https://colinraffel.com/projects/lmd/

3.3. Semi supervised learning

The G2G Lakh model is trained on the triplets thus created which constitutes a semi-supervised training. In order to simplify the problem we don't take into account the velocity of the notes (sound power of the notes, force with which the electronic keyboard key is pressed). As the content between 2 different segments of a music is probably different, we want to teach the model to transfer the style of the drums from the second segment to the first one, taking into account the content of the instrument tracks. The learning is performed on 1M triplets, i.e. about 1.3 times the synthetic train data.

3.4. Supervised learning

In order to check if the Lakh dataset improves the style transfer of drums compared to the synthetic base, we also train G2G Lakh on the synthetic base in a supervised fashion, taking into account only the drums. To be consistent with the Lakh experience, a similar number of triplets are used and velocity is not taken into account. The model and the parameters are the same between the 2 experiments, only the database changes: semi-supervised learning on the Lakh database and supervised learning on the synthetic database.

4. Objective evaluation metrics

We use the above models to perform style transfer between drums sequences. For that, we feed the networks with pairs of $(content, style)$ songs where the style input is a drums sequence. We expect the models to generate in output drums sequences that share the same style with the corresponding style input, and that preserve some information from the content input. We want also to track some eventual properties like repetitiveness of some groove pattern in the generated sequences.

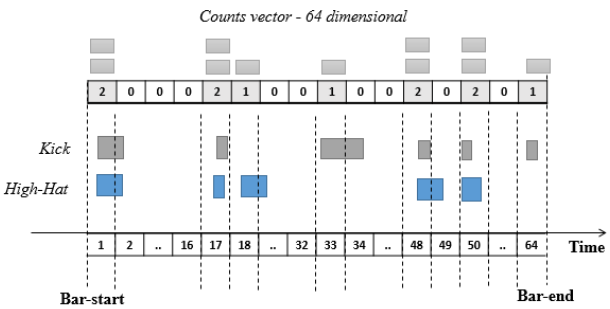

Fig. 5. Notes density vector representation of a (drums) bar.

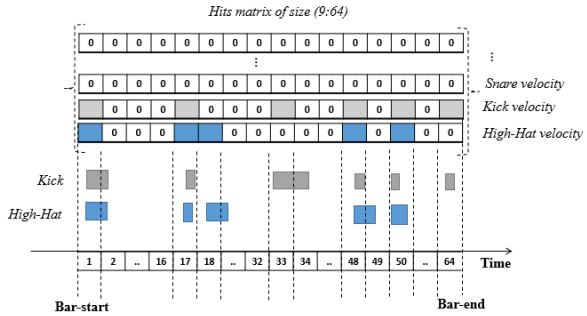

Fig. 6. Hits matrix representation of a (drums) bar with Kick and High-Hat kits.

In this section we define and motivate the use of some objective metrics that would help us assess the structural properties of generated drums and the quality of style transfer task with respect to content (similarity between content input and the output) and style (style fit between style input and the output).

4.1. Tracking repetitiveness in generated drums

We often assume that real drums performances are repetitive. Here, we propose a bar-level tracker that helps measure the extent of similarity between two bars of drums notes in terms of some groove pattern (4). A first variant focuses on repetitiveness of played notes densities through the compared bars. The second one includes notes velocities and encodes notes from each drum kit separately, and tries instead to measure repetitiveness of velocity-based patterns through the compared bars.

4.1..1. Bar-level similarity, in terms of notes density

The bar is divided into 64 equal-sized time segments. We represent a bar with a 64 dimensional vector. Each dimension $1 \leqslant i \leqslant 64$ holds the number of times there were note onsets on that segment of the bar (Fig. 5). We can then normalize the obtained vector to obtain a probabilistic equivalent representation. Such a vector represents the structure captured by quantification of time axis, giving 64 discrete reference time positions, and describes how bar's notes are distributed between them. We refer to this structure as the groove pattern of the bar in question. The extent of similarity between two bars could then be measured as the Battacharya similarity, computed on their corresponding probability vectors ($\in [0, 1]$) and it should tell how well is the groove pattern of one bar correlated with the other.

4.1..2. Bar-level similarity, in terms of notes velocities

The bar is always divided into 64 equal-sized time segments, but is now represented with a hits matrix of size (9, 64). The first dimension identifies the drum kit, and the second one corresponds to quantification of time axis (Fig. 6). For a given drum kit, the corresponding line in hits matrix holds played notes velocities (averaged). Such a matrix captures again a new form of groove pattern, velocity-based this time. The extent of similarity between two bars could then be measured as the Shannon-Jensen entropy, computed on their corresponding flattened hits matrices ($\in [0, 1]$) and it should also tell how well is the groove pattern (in this new sense) of one bar repeated in the second.

4.1..3. Definition of piece-wise similarity

We can extend the use of the above bar-level similarity scores to sequences in two possible ways. Given a sequence D, we can define the intra-sequence groove similarity of D as the average bar-level groove similarity, computed on all possible pairs of different bars that come from D. It tells how likely is the sequence D to follow some repetitive groove pattern in one of the above senses. Or, given two sequences A and B, we can define the inter-sequences groove similarity between A and B as the average bar-level groove similarity, computed on pairs of synchronized bars from A and B. It should be able to tell how likely the sequence A and the sequence B are to share similar groove patterns in one of the above senses.

4.1..4. Efficiency of piece-wise similarity on real drums

We test the proposed piece-wise similarity score in the intra-sequence fashion on examples of real drums sequences from Groove2Groove synthetic dataset. To confirm the hypothesis that real drums are repetitive in this dataset, and to get a feeling of how repetitive are they, we compute groove similarity scores over a set of 2400 MIDI drums sequences. We should expect high scores, and this is empirically verified in Fig. 7, saying that real drums are really repetitive in some sense. On the other side, sequences of non-drums notes should in general be less repetitive, and this is again empirically checked using the piece-wise similarity score computed on non-drums tracks as shown on the same figure. This demonstrates the pertinence of the metric and motivates its use to evaluate the generated drums by our models in the next section. The piece-wise similarity score could serve

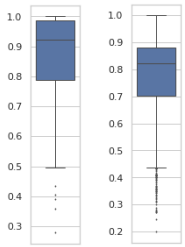

Fig. 7. Piece-wise similarity score (in terms of notes densities) computed on a set of 2400 sequences. Left: drums, highly repetitive groove pattern. Right: non-drums, less repetitive.

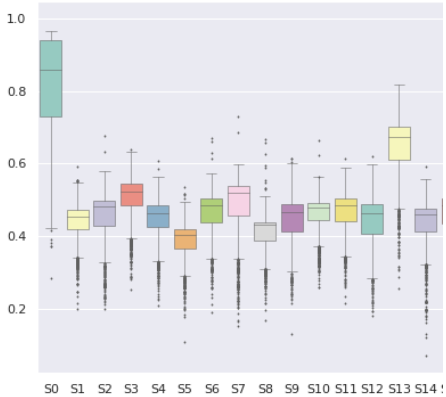

Fig. 8. Piece-wise similarity score (in terms of notes densities) computed in inter-sequences fashion over pairs of different styles. One style is confronted to 15 different styles from Groove2Groove synthetic dataset.

to track the repetitiveness within generated drums, in one of the previous senses.

It also comes out that the proposed piece-wise similarity score (in terms of notes densities) is a good indicator of how such patterns are implied in the style transfer process. We computed the piece-wise similarity score, in the inter-sequences fashion, between pairs of sequences that come initially from the same style. The obtained scores were relatively high for pairs that initially share the same style (Fig. 9), and less important otherwise (Fig. 8). This proves that the metric is useful to capture style matching/difference between pairs of drums sequences. Moreover, it tells that when two drums sequences share the same style (as observed in this particular dataset), they're more likely to share some repetitive groove pattern, in terms of notes densities. Maybe this aspect is one of the things that are being effectively transferred during the style transfer process. In the next section, we will be assessing this property within the generated outputs in comparison with their style inputs: they're supposed to share same style and may consequently benefit from high similarity scores in the above sense i.e the style transfer process is taking into account the repetition of some groove pattern (related

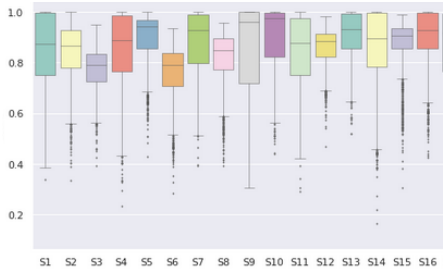

Fig. 9. Piece-wise similarity score (in terms of notes densities) computed in inter-sequences fashion over pairs of the same style. 16 different styles are investigated form Groove2Groove synthetic dataset.

to notes densities) between the generated sequence and style input. Note that the same property can be verified and tracked using the metric in terms of notes velocities, as demonstrated in figures Fig. 22 and Fig. 23 of the appendix.

4.2. Assessing the quality of transfer task

We want to address the problem of measuring how well the style transfer task was performed. Two natural aspects are to consider at this level: how similar is the generated output with respect to content input, what structures are then retained ? And how well is the style fit between style input and the generated sequence, both supposed to share the same style, what's being transferred then ?

4.2..1. Style fit

How to measure if the drums track generated by our model is in the same style as our style track ? We suggest 4 metrics of similarity between two different drums tracks.

4.2..1..1. Onset Drums Taken from Groove2Groove article, the histogram of the sequence is constructed with in abscissa the notes' start time modulo one bar and in ordinate the pitch of the note (the type of percussion). The similarity value between 2 sequences is the cosine similarity value of the 2 flattened histograms. Checks if the same percussion instruments fall at the same position in the bar. Gives a value between 0 and 1, better if it is close to 1.

4.2..1..2. Onset Cosine similarity between the histograms of the beginning of notes modulo one bar of the 2 sequences. Checks if the percussions fall at the same place in the bar, statistically, independently of the type of percussion. Has less information than onset-drums but observes the rhythm globally (3).

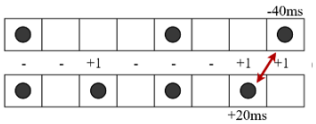

Fig. 10. Hamming distance with micro-adjustment illustrated.

4.2..1..3. Hamming distance with micro-adjustment For two symbol sequences of the same length, the Hamming distance associates the proportion of positions where the two sequences differ. We use the GrooveToolbox (1) library which proposes measures of similarity between drum tracks. The "Fuzzy Hamming" distance extends the standard Hamming distance by penalizing less the very slight differences between 2 drum tracks: with the standard distance, if 2 notes do not fall exactly at the same time, there are 2 points of disagreement between the tracks (a note is present on one track and not the other, then the opposite); with the micro-setting, if a note present on one track is absent on the other, we look to see if it falls slightly before or after (within a certain limit) and we weight by the distance.

4.2..1..4. "Structural Similarity" distance The Hamming distance penalizes notes that are played in similar but not identical positions. GrooveToolbox offers the implementation of a measure of the similarity of drum track structure: using a standard metric profile that defines "strong" and "weak" positions in a beat, notes can be considered "ornaments" based on the metric profile and the notes surrounding them. Removing these ornaments results in a simplified rhythmic representation of the drum track, analogous to its "structure". We then calculate the standard Hamming distance between 2 structures. The article shows that this measure is relevant to compare 2 drum tracks, as well as Fuzzy Hamming distance.

4.2..2. Similarity with content

If it is not too complicated to measure the similarity of 2 drum tracks (a simple Hamming distance between two binarized Piano roll drums being already very relevant) and therefore to measure the similarity between a drum track generated by our model and the reference style (which is also a drums track), it is more difficult to check if this generated drums track is well in adequacy with the content. It is necessary to verify that the instrument tracks of the content have an impact on the output, and that this impact is significant and results from a certain form of harmony between the content and the output.

An intuitive solution, but of limited relevance, would be to compare the generated drums track with the drums track of the content (if the content includes drums, which will be the case in our test databases). We would then use metrics to evaluate the style, which are relevant as we have shown. However, the style of the content is probably different, even very different from the chosen style. We therefore expect to have quite low similarity metrics, but if the drums capture the structure of the content, the comparison of the structure of the drums tracks may be relevant. A second solution would be to directly measure the "fit" (or extent of similarity) between the content instruments and the generated drums. For that, two metrics are proposed.

4.2..2..1. Correlation of notes density We count the number of notes in a sequence (8 bars in our case) by dragging a sliding window of one bar long. On the content track, we count the number of notes played by all the instruments, independently of the instrument, and on a drums track we do the same with the percussion instruments. It is assumed that the note density of the drum track should follow (correlate with) the note density of the instrument tracks. The Pearson correlation coefficient of the two note densities (content and output) is calculated and gives a value between -1 (negatively correlated) and 1 (positively correlated) or 0 represents a non-correlation.

4.2..2..2. Comparison of self-similarity matrices A self-similarity matrix (SSM), for a sequence of tokens $[E1, ..., En]$, is an array of dimension (n, n) in which the value at index (i, j) is equal to the similarity between the elements Ei and Ej. It is a symmetric matrix, the measure of similarity being symmetric and the diagonal elements have a similarity equal to 1 (similarity with respect to themselves). We represent the sequences in the form of a piano roll by sampling at the quarter beat and discretizing the piano roll (1 if at least 1 note present at a given pitch and at a given time, no matter if there are several notes or not and without considering their velocity, 0 if no notes). To reduce the size of the matrix, we remove unnecessary rows (unused pitches) and we combine 4 columns into 1: we concatenate the 4 vectors that make up 1 beat into 1 vector, 4 times longer. Our element sequence is therefore a sequence of "beats" represented by a binary vector. As a measure of similarity we use the standard Hamming distance. By calculating the SSM we check if the same notes are played at the same time in 2 different bars. The self-similarity matrix highlights the patterns that are repeated in a musical sequence and we can extract information such as the structure of the piece or determine the melodies that are repeated, for example. In the figure below we see in figure 12.a) the SSM of Michael Jackson's Billie-Jean instruments, in 12.c) the SSM of the drums of the piece, which are highly repetitive. In 12.b), the drum track generated by the G2G Lakh model inspired by the style of Bob Marley's Exodus: we recognize a structure similar to that of the instrument tracks, which suggests that the model takes into account the content to generate the drum track. It is the similarity of these structures that we try to evaluate with this second metric. The repetitive patterns in a musical sequence are represented by diagonal lines in the self-similarity matrix. If you have a repetition of a beat further in the content sequence, you can expect that same beat to repeat itself in the drums track as well (pattern synchronization between content and drums). Simply put, the "white dots"

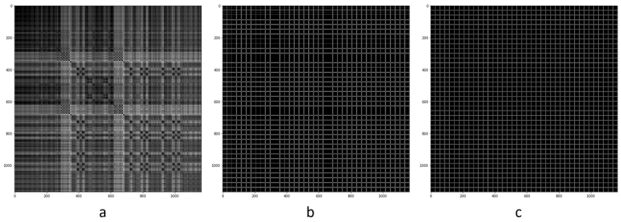

Fig. 11. Self-similarity matrices for the instruments (a) and drums (b and c) of a piece of music.

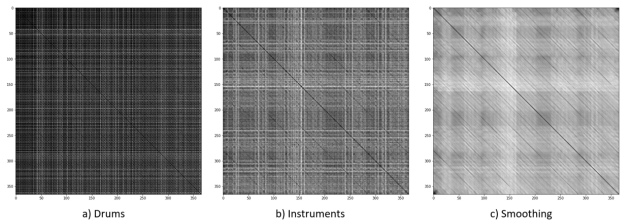

Fig. 12. Illustration of the smoothing process of SSM.

(self-similarity close to 1) in the content and the drums should be placed in the same place. We threshold the SSM of the content (0.95) to keep only the beats with strong self-similarity (see figure 13.e)). After having calculated the SSM of the content (SSM content) and the output drums (SSM drums), we propose the following measure: Similarity = ratio(SSM content X SSM drums) / ratio(SSM content) Where X is the point by point multiplication and ratio represents the number of "white points" divided by the total number of points in the matrix. We obtain a number between 0 and 1: the closer it is to 1, the more the repetitive structures are synchronized.

4.2..2..3. Smoother version with SSM The Libfmp library [3] offers tools for music analysis and in particular for music structure analysis. A function of this library is used to horizontally smooth the result of the SSM of the content (see figure 13.c) above) and a function to threshold the result obtained (figure 13.d), highlighting the repetitive melodies of the content. The same calculation is then performed as before from this content matrix.

[3] https://www.audiolabs-erlangen.de/resources/MIR/FMP/C0/C0.html

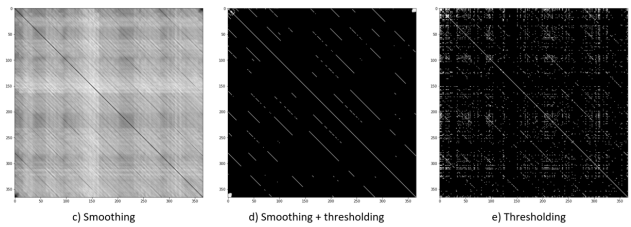

Fig. 13. Illustration of the thresholding process of smoothed SSM.

5. Analysis of generated drums

We used the above objective metrics during the analysis of drums sequences generated by our models to draw empirically some remarks about the structure/quality of outputs.

5.1. Generated drums are mostly repetitive

The first observation is that all models are generating drums sequences that are too repetitive in the sense of the piece-wise similarity score presented in 4.1.4, namely in terms on notes densities. Our models are generating drums sequences whose bars share some repetitive groove pattern in terms of played notes densities. This can be verified on figure 14. We can so far assume that G2G Lakh and G2G Plus are more determined about repetitiveness than G2G Basic, but all models are considerably repeating some groove pattern through the bars of the generated sequence.

The attention weights confirm this hypothesis of repetitiveness being part of the style transfer process. In fact, it turns out that all our models are attending to almost only the first parts of the content song, in a very repetitive way. Following the visualizations in figures 15, we can expect highly repetitive sequence of tokens in output. And this might for instance explain the high similarity scores obtained in figure 14. It also helps explain the results observed later in 5.2 with respect to style.

5.2. Repetitive groove patterns are potentially part of the transferred style

We compute similarity scores in terms on notes densities over pairs of generated drums and their corresponding style inputs (also drums). According to paragraph 4.1.4 and since each pair's sequences are supposed to come from the same style, we should expect relatively high scores, like in figure 9. And if so, we can say that the tracked groove pattern is somehow implied in the style transfer process. This was empirically verified as shown on figures 24, 25, and 26. We observe relatively high scores, not as good as in figure 9 but more important than the opposite

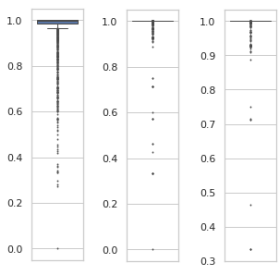

Fig. 14. Piece-wise similarity score (in terms of notes densities) computed in intra-sequence fashion over generated drums sequences. From left to right: G2G Basic, G2G Lakh, G2G Plus.

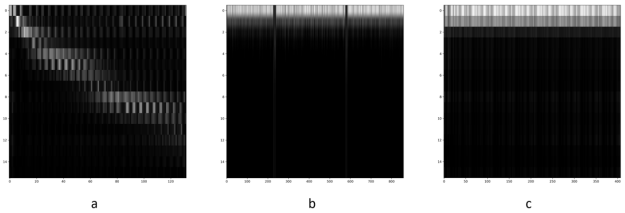

Fig. 15. Visualization of attention weights. Horizontal axis: output tokens, vertical axis: hidden states of the content. Values of the matrix are the weights. (a) G2G Basic. (b) G2G Lakh. (c) G2G Plus.

case ones (sequences from different styles) shown on figure 8. We can so far assume that our models are implying such groove information in the style transfer process, in a repetitive way. In other words, generated drums bars are more likely to share some groove pattern with the style input bars, in the sense of the used metric.

5.3. Similarity to content and style fit

5.3..1. Preliminary results

After listening to many outputs, we could observe that the results produced by the G2G Plus and G2G Lakh models are better than those of the original G2G Basic model trained on all instruments: the drums produced have more repetitive patterns, the percussions' notes seem less random and the global rendering looks more like a drum track. The style seems to be well preserved. However, it is more difficult to evaluate the adequacy with the content when listening to it: if the drums track sounds rather well when you listen to it when remixed with the other instruments, the silences and phase changes do not seem so synchronized (for example, a percussion pattern can be the same without change when you go from a verse to the chorus). To check if the decoder, when generating the drums track, takes into account the content (coming from the content encoder), we can observe the attention matrix that is used to achieve a kind of alignment between the hidden states of the content's encoder and the decoder.

Again, we refer to the attention weights shown on figure 15 fetched while transferring the style of Marley - Exodus to the content of Michael Jackson - Billie Jean. Segment of 8 bars of the drums track. On the horizontal axis: tokens generated successively by the decoder. On the vertical axis: successive hidden states of the encoder, starting from the top. We consider a segment of 8 measurements that is why there are 16 hidden states (2 vectors per measurement at the input of the encoder GRU). We notice that the original G2G Basic model seems to take into account the content of the instrument tracks (as the tokens are generated, it takes into account deeper and deeper hidden states of the encoder: there is an alignment between the input and output sequences) but the 2 other models only consider the first

hidden states of the encoder content. The other hidden states are not totally ignored but their impact is much smaller. For the G2G Lakh model for example, the drums track seems to be generated only by taking into account the first 2/3 hidden states of the content encoder, i.e. the first 6 seconds of the song at 60 BPM (out of the 32 seconds in total). This may be enough to get enough information to generate a drums track 8 bars long, all the more so since the models sometimes look for deeper hidden states (cf. the 2 vertical white stripes on model b and the gray areas on the deeper hidden states of model c). Moreover, model a) generates drums badly and its drums generation scheme is the one that allows it to generate the other instruments, for which the supervised data have very aligned sequences between input and output, which creates this linearity between input and output in the attention matrix.

5.3..2. Evaluation results on synthetic test set

The synthetic database was created for the original Groove2Groove article in the following way: we selected 20 styles of Band-in-a-box never used before (in the training and validation sets) and for each of these styles, we generated about 500 measures of music (from the original chord table). Each style contains 2 sub-styles considered as different styles, so we have 40 styles in total. The synth test set therefore contains 2400 triplets (source, style, target). The advantage of this dataset is that it contains for each pair (content, style) a target sequence resulting from the ”style transfer” (generated by Band-in-a-box from the content, in the selected style). All tracks contain drums. It is therefore relevant to compare our results with the target. This will not be possible with the Lakh database afterwards, which obviously does not contain a target track.

We have 4 models to evaluate :
- The original Grv2grv model used in the article: trained on synth on all instruments, without the velocity (our reference, we try to produce better drums than this model). In the results of the evaluation we will call it Grv2Grv
- The same model, also trained on synth in a supervised way, but keeping only the drums in style and target (trained only on drums, without velocity): Grv2Grv drums
- The model of section II. with drums encoder trained in a supervised way on synth, only on drums (with velocity): Grv2Grv+drums
- The Lakh model, the same model as Groove2Groove but trained in a semi-supervised way on the Lakh base, as on the drums, without velocity: Lakh

The 2 points we have to evaluate are fidelity to the content and similarity with the style. We evaluate the 4 models in 2 modes: greedy (we retain the token with the highest probability at each stage of decoding/generation) and sampling (we choose at each stage a token according to the softmax distribution, after dividing the logits by a temperature parameter t to control their randomness). We take t=0.6 as in the article because this parameter would give good results.

5.3..2..1. Similarity between the drums track generated by the models and the drums track of the style figure 16 gathers the Hamming distance and Structural similarity measurements: the best scores are obtained for lower distance values. The right plot gathers the measures of similarity: onset.drums, onset and Bhattacharyya distance. The best models are those with the closest average similarity to 1. The Grv2Grv drums model, which has been trained on the synthetic basis only on drums, produces the best results for the 5 measures if we look at the median (4 in greedy mode and 1 in sampling mode), even in appendix table where the best performances are in bold. This result was expected because the model is evaluated on the same database as the one on which it was trained (synth), by simply changing the styles.

5.3..2..2. Similarity between the drums track generated by the models and the drums track of the content The same assessment is made as before but in relation to the drums track of the content: if the drums are in agreement with the instruments, it is expected that their structure will be correlated with that of the content. Thus, to evaluate if the generated drums track is in agreement with the content, it can be interesting to compare its structural similarity with the content drums track. In figure 18, we notice that the Lakh greedy model produces the best results for all measures, whether we look at the mean or the median.

5.3..2..3. Similarity between the drums track generated by the models and the drums track of the target Finally, the target track gives a good indication of the expected result: generated by BIAB with the same content, in the chosen style. During the supervised training, it was the ground truth resulting from the style transfer, for the drums. The results produced by Grv2Grv drums are very close to this target: considering the median, Grv2Grv produces the best results for 4 measures out of 5 (4 measures in greedy mode and 1 in sampling mode). The original model produces the best results for 2 measures if we look at the mean. Obtained scores ares shown on figure 17.

5.3..3. Evaluation results on Lakh dataset

Lakh’s music is very diverse because it is a transcript of existing music, which can be of any register. We kept 15% of the data from the Lakh database to test our models. Since the musics are named by their MD5 checksum, the musics in our test set are completely random, so we expect to encounter many styles never seen in the rest of the database, allowing us to test Grv2grv’s ability to transfer a style in ”One-shot”, on styles never seen before. To create the peers (source, style) from the MIDI files, preprocessing is done again, keeping only the 8-bar segments that contain at least 64 percussion notes (2 notes

per beat on average) and at least 64 instrument notes. This allows you to have tracks where instruments and drums are significant. In the end, it's not very discriminating compared to our first preprocessing since we keep 11,750 MIDI files, or 126,904 segments. For the evaluation we randomly take 5000 pairs of segments without replacement.

5.3..3..1. Similarity between the drums track generated by the models and the drums track of the style Looking at the mean and the median in figure 19, the Lakh greedy model produces better results on Hamming distance and Structural similarity measurements. The 3 other measurements are won by the Grv2Grv (sampling) model. Grv2Grv drums follows these models on several measurements.

5.3..3..2. Similarity between the drums track generated by the models and the drums track of the content Following figure 20, the Lakh greedy model wins 3 out of 5 measures (Hamming distance, Structural similarity and onset.drums) for similarity with the content drums track and the Grv2Grv sampling model the 2 remaining measures (onset and Bhattacharyya), looking at the mean or median. Finally, as can be seen in figure 21, right-hand graph, the palm of similarity with the instrument tracks of the content is shared between the different models, with Grv2Grv sampling standing out on the Note Density Correlation measure.

6. Conclusion

We wanted to improve the generation of drums in a given style, to accompany a given content and for that we proposed different models derived from Groove2groove, trained in different ways. Groove2groove ignored the drums of the content so we added a specific encoder to take them into account. When generating the output tracks, there was no indication that the track to be generated was a drums track and not an instrument track, resulting in an unsatisfactory accompaniment. We therefore chose to simplify the problem by only training our models to generate drums, to solve this representation problem and at the same time reduce the complexity of the task. To evaluate the drums generated by our models, we proposed new metrics to compare the adequacy between 2 percussion tracks, and 1 instrument track vs. 1 drums track. The results of the evaluation do not allow us to declare that one model differs greatly from the others, but the Grv2grv drums and Lakh models, trained respectively in a supervised and semi-supervised way only on drums (taking into account the content instruments) and without using velocity (to simplify the problem) produce notable results, in terms of the number of measures they dominate. Making the training specific to drums therefore improved the results as expected. Moreover, on new styles derived from real music, Lakh produces good results in terms of fidelity to style and content, showing that the synthetic base may not be diverse enough compared to existing music. We were able to evaluate whether the generated drums are close to the desired style, but we should be able to evaluate more precisely the correlation between drums and instrument tracks.

7. Perspectives

In order to compensate the problem of attention weights being focused on the very beginning of content sequence, we can for instance change the attention mechanism implemented in Groove2Groove. Initially, it uses Bahdanau attention. We can instead use Local (a smoothed version of Hard attention) attention to force the weights to attend for specific regions of content sequence while generating the output. This might help solve the problem of attention weights being concentrated only on first hidden states as observed previously.

The complexity of the attention matrix limits the size of the sequences used to train Groove2groove. In order to take into account the real complexity of the music pieces, their alternating structure of verses, choruses, bridges, pre-refrains etc. and the long-term dependencies of the notes, it would be necessary to use complete music and thus rethink the attention mechanism.

Finally, we realized how difficult it is to evaluate with objective measures the quality of a musical sequence, or a percussion sequence mainly in our case. The quality of a music can not be better evaluated than the human ear, that's why we realized a website [4] allowing to realize style transfers and including examples of generated pieces. To determine which model is the most promising, users should be asked to judge because, even if a piece of music can meet certain measurable criteria, its quality remains subjective.

8. Appendices

[4] https://prim-project.000webhostapp.com/

ON SYNTHETIC DATASET

	Hamming	S. Similarity	onset.drums	onset	Bhattacharyya	Note density	SSM	SSM Smoothed
CONTENT	**7.0 — 7.0552**	4.2426 — 4.2064	0.0547 — 0.1315	0.5456 — 0.4813	0.3066 — 0.3625	0.1632 — 0.174	0.9732 — 0.9586	0.9038 — 0.8875
Grv2Grv (greedy)	**5.099 — 5.2406**	3.1623 — 3.1845	**0.3457 — 0.3668**	**0.6594 — 0.5322**	**0.4735 — 0.4607**	**0.1331 — 0.0969**	0.9597 — 0.9523	0.9205 — 0.9154
Grv2Grv (sampling)	5.5678 — 5.6928	3.3166 — 3.2886	**0.4478 — 0.427**	**0.7727 — 0.5679**	**0.5701 — 0.4891**	**0.6474 — 0.5496**	0.9413 — 0.9349	0.8935 — 0.8856
Grv2Grv+drums (greedy)	6.33 — 6.3932	3.873 — 3.8412	0.1438 — 0.2697	0.5397 — 0.4871	0.3729 — 0.3619	-0.0012 — -0.0511	0.9643 — 0.9512	**0.9369 — 0.9282**
Grv2Grv+drums (sampling)	**6.8557 — 6.8084**	**3.7417 — 3.807**	0.2911 — 0.3377	0.681 — 0.5359	0.392 — 0.3654	-0.0384 — -0.0382	0.9661 — 0.958	**0.9415 — 0.9305**
Lakh (greedy)	**5.099 — 5.2332**	**3.0 — 3.0492**	0.3351 — 0.3617	0.7314 — 0.5537	0.4901 — 0.459	0.0675 — 0.0758	0.9643 — 0.9493	0.9046 — 0.8932
Lakh (sampling)	5.6569 — 5.7473	**3.3166 — 3.2352**	0.3048 — 0.3357	0.6982 — 0.5454	0.4444 — 0.4354	0.0462 — 0.0455	**0.9659 — 0.95**	0.9101 — 0.8979
Grv2Grv_drums (greedy)	5.2915 — 5.3235	**3.0 — 3.0508**	0.3705 — 0.396	0.6876 — 0.5397	0.4741 — 0.4453	0.0765 — 0.0581	**0.9701 — 0.9595**	0.9279 — 0.9192
Grv2Grv_drums (sampling)	5.5678 — 5.6087	**3.0 — 3.0828**	0.4151 — 0.4165	0.7272 — 0.5533	0.4767 — 0.439	0.0302 — 0.0225	0.97 — 0.9604	0.9337 — 0.9234
STYLE						0.0282 — 0.024	0.9583 — 0.9448	0.9002 — 0.883

STYLE CONTENT NOTES

	Hamming	S. Similarity	onset.drums	onset	Bhattacharyya
CONTENT					
Grv2Grv (greedy)	**6.4031 — 6.502**	4.0 — 3.9791	**0.0402 — 0.1502**	**0.5982 — 0.4997**	**0.4144 — 0.4107**
Grv2Grv (sampling)	6.9282 — 6.9988	4.1231 — 4.1051	**0.0858 — 0.1747**	**0.7037 — 0.5325**	**0.4783 — 0.4336**
Grv2Grv+drums (greedy)	7.0711 — 7.0924	4.3589 — 4.3272	0.0289 — 0.1245	0.535 — 0.4751	0.3583 — 0.3421
Grv2Grv+drums (sampling)	**7.6577 — 7.5732**	**4.3589 — 4.3273**	0.0717 — 0.1556	0.6631 — 0.5242	0.3828 — 0.3487
Lakh (greedy)	**6.0828 — 6.2073**	**3.6056 — 3.6052**	**0.1133 — 0.1949**	0.6859 — 0.5245	0.4473 — 0.4271
Lakh (sampling)	6.6332 — 6.6791	**3.7417 — 3.7797**	0.0971 — 0.1815	0.6639 — 0.5243	0.3799 — 0.4047
Grv2Grv_drums (greedy)	6.6332 — 6.6933	**4.0 — 4.0083**	0.0619 — 0.1573	0.6372 — 0.5085	0.4249 — 0.4023
Grv2Grv_drums (sampling)	7.0 — 7.0276	**4.1231 — 4.0605**	0.0752 — 0.1664	0.6692 — 0.5221	0.4276 — 0.3968
STYLE	7.0 — 7.0552	4.2426 — 4.2064	0.0547 — 0.1315	0.5456 — 0.4813	0.3066 — 0.3625

CONTENT

ON LAKH DATASET

	Hamming	S. Similarity	onset.drums	onset	Bhattacharyya	Hamming	S. Similarity	onset.drums	onset	Bhattacharyya
CONTENT	**6.6061 — 6.5557**	4.0 — 4.0242	0.2563 — 0.3114	0.9089 — 0.892	0.7328 — 0.715					
Grv2Grv (greedy)	**4.0 — 4.0667**	2.2361 — 2.3522	**0.8141 — 0.7704**	**0.9512 — 0.9164**	**0.8786 — 0.8374**	**6.1644 — 6.2004**	3.873 — 3.9168	0.2624 — 0.3089	0.9055 — 0.8743	0.8004 — 0.7531
Grv2Grv (sampling)	4.2426 — 4.4936	2.4495 — 2.4568	**0.8233 — 0.7922**	**0.951 — 0.9282**	**0.852 — 0.8162**	**6.5574 — 6.6413**	4.0 — 4.047	0.2712 — 0.3174	0.9104 — 0.8885	0.7777 — 0.7426
Grv2Grv+drums (greedy)	4.1231 — 4.186	2.4495 — 2.4523	0.8249 — 0.7554	0.9529 — 0.9027	0.8623 — 0.8105	6.3246 — 6.3077	4.0 — 4.0239	**0.2444 — 0.2998**	0.9034 — 0.8661	0.7467 — 0.7171
Grv2Grv+drums (sampling)	**4.4721 — 4.4817**	**2.4495 — 2.5152**	0.8304 — 0.7876	0.9536 — 0.9196	0.8228 — 0.7825	6.7082 — 6.6244	4.0 — 4.0697	**0.2575 — 0.3112**	0.9107 — 0.8847	0.7091 — 0.6964
Lakh (greedy)	**4.1231 — 4.249**	**2.4495 — 2.5276**	**0.6747 — 0.6217**	0.9369 — 0.9236	0.8596 — 0.811	**5.9161 — 5.9296**	**3.6056 — 3.6509**	**0.3814 — 0.3728**	**0.9138 — 0.899**	**0.8188 — 0.7848**
Lakh (sampling)	4.7958 — 4.9153	**2.8284 — 2.7975**	0.6263 — 0.5831	0.9245 — 0.9053	0.8388 — 0.7936	6.245 — 6.2682	**3.7417 — 3.7872**	0.3521 — 0.3519	0.9062 — 0.8855	0.801 — 0.7552
Grv2Grv_drums (greedy)	**3.4641 — 3.6695**	**2.0 — 2.169**	0.8582 — 0.7959	**0.9572 — 0.9101**	**0.8854 — 0.8331**	6.1644 — 6.1479	**3.873 — 3.9349**	0.2448 — 0.3045	0.8945 — 0.8658	0.7655 — 0.7315
Grv2Grv_drums (sampling)	3.7417 — 3.8959	**2.118 — 2.2077**	**0.8614 — 0.8081**	0.9561 — 0.9194	0.8468 — 0.8143	6.4031 — 6.4366	3.873 — 3.9898	0.2525 — 0.3106	0.9019 — 0.878	0.7357 — 0.7185
TARGET	2.0 — 2.2788	1.4142 — 1.295	0.9847 — 0.9685	0.9943 — 0.9853	0.8928 — 0.861	6.5574 — 6.507	3.873 — 3.9877	0.26 — 0.3154	0.9133 — 0.8981	0.8461 — 0.7976
STYLE							6.6061 — 6.5557		0.9089 — 0.892	0.7328 — 0.715

STYLE CONTENT

	Hamming	S. Similarity	onset.drums	onset	Bhattacharyya	Note density	SSM	SSM smoothed
CONTENT	6.5574 — 6.507	3.873 — 3.9877	0.26 — 0.3154	0.9133 — 0.8981	0.8461 — 0.7976	0.2284 — 0.2925	0.9722 — 0.9606	0.9131 — 0.9122
Grv2Grv (greedy)	4.0 — 4.0784	2.4495 — 2.3724	0.8154 — 0.772	0.9516 — 0.9187	**0.8945 — 0.8522**	**0.1844 — 0.1505**	0.95 — 0.9472	**0.9133 — 0.9072**
Grv2Grv (sampling)	4.3589 — 4.507	2.4495 — 2.4744	0.8233 — 0.7927	0.9531 — 0.9297	0.8709 — 0.8317	0.082 — 0.0744	0.9519 — 0.9471	0.9008 — 0.8955
Grv2Grv+drums (greedy)	4.1231 — 4.1937	2.4495 — 2.4664	0.8242 — 0.755	0.9542 — 0.9047	0.8698 — 0.8159	0.1157 — 0.0671	0.9516 — 0.947	0.9076 — 0.9017
Grv2Grv+drums (sampling)	4.4721 — 4.5012	2.4495 — 2.534	0.8299 — 0.7866	0.955 — 0.9206	0.8345 — 0.789	0.0145 — 0.0037	0.9543 — 0.9491	0.9037 — 0.8983
Lakh (greedy)	4.1231 — 4.2594	2.4495 — 2.5508	0.6716 — 0.6218	0.9367 — 0.924	0.8608 — 0.8123	0.1409 — 0.1396	0.9358 — 0.9329	0.8874 — 0.8813
Lakh (sampling)	4.7958 — 4.9346	2.8284 — 2.8269	0.6265 — 0.5817	0.9232 — 0.9052	0.8421 — 0.7959	0.0792 — 0.0688	0.9537 — 0.9438	0.8977 — 0.8895
Grv2Grv_drums (greedy)	**3.4641 — 3.6854**	**2.0 — 2.1816**	0.861 — 0.7996	**0.9582 — 0.9132**	0.8929 — 0.843	0.1172 — 0.086	0.9536 — 0.9499	0.9092 — 0.9036
Grv2Grv_drums (sampling)	3.7417 — 3.9248	2.2361 — 2.2297	**0.8613 — 0.8078**	0.9565 — 0.9204	0.8587 — 0.8215	0.0698 — 0.0554	**0.9559 — 0.9509**	0.9059 — 0.897
TARGET						0.2426 — 0.2956	0.9761 — 0.967	0.9196 — 0.9155
STYLE	2.0 — 2.2788	1.4142 — 1.295	0.9847 — 0.9685	0.9943 — 0.9853	0.8928 — 0.861	0.0143 — 0.0115	0.9659 — 0.9592	0.9194 — 0.915

TARGET CONTENT NOTES

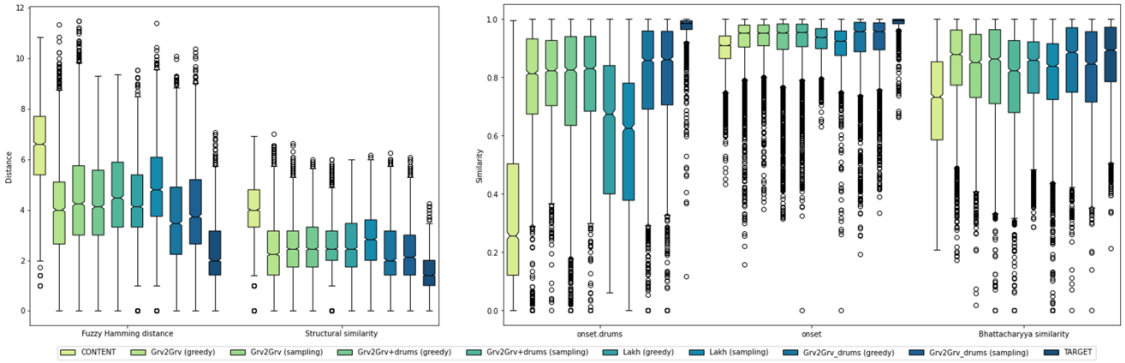

Fig. 16. Similarity between the generated drums track and the style drums track.

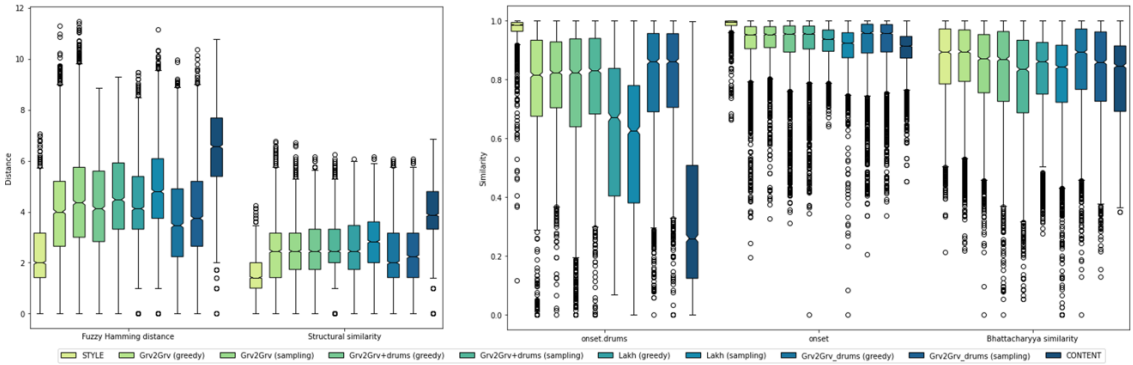

Fig. 17. Similarity between the drums track generated by the models and the target's drums track.

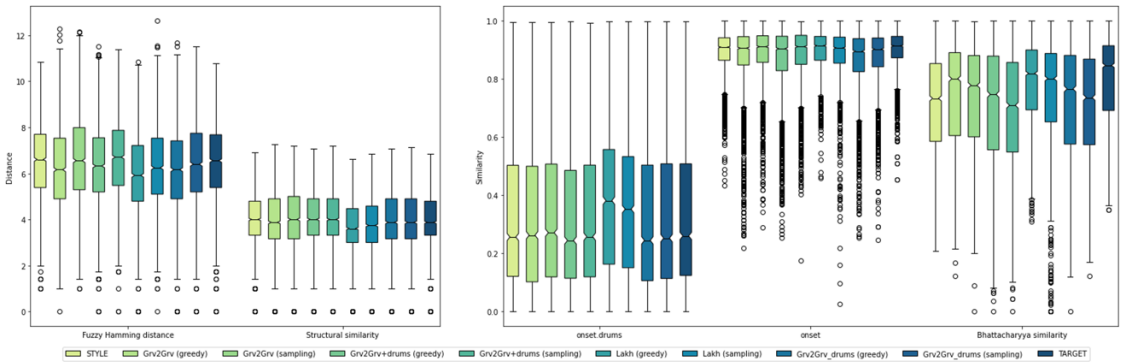

Fig. 18. Similarity between the drums track generated and the drums track of the content.

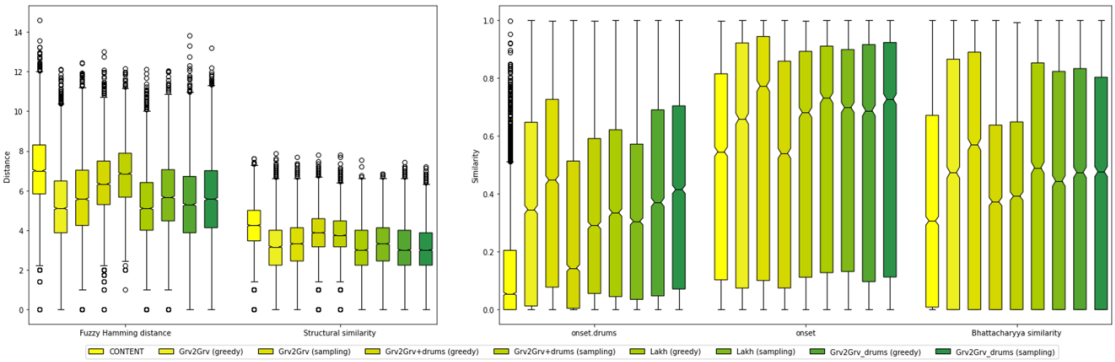

Fig. 19. Similarity between the generated drums track and the style drums track.

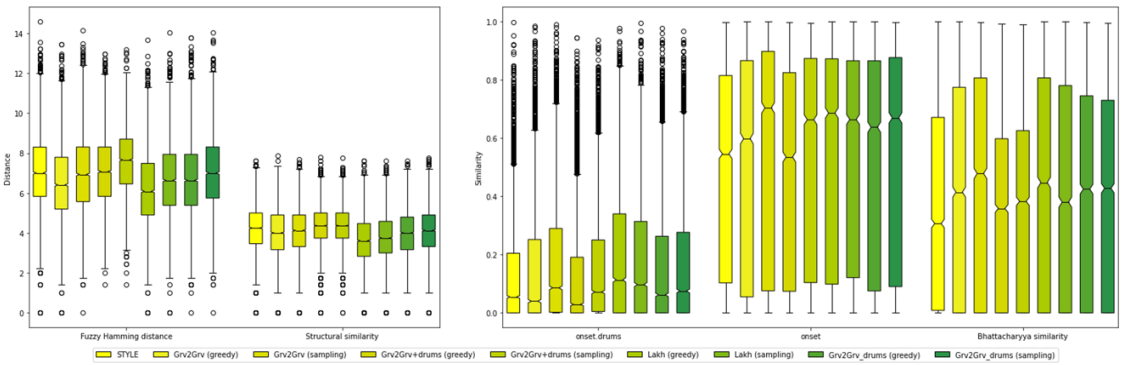

Fig. 20. Similarity between the drums track generated and the drums track of the content.

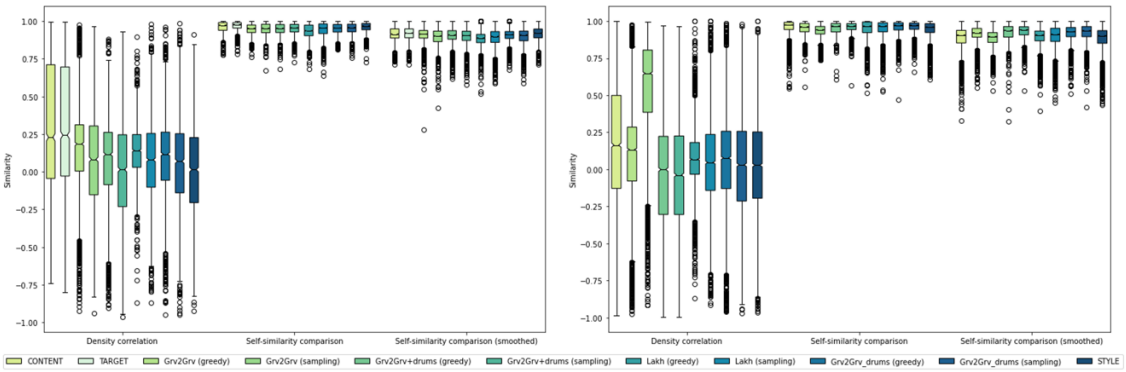

Fig. 21. Similarity between the generated drum track and the notes of the content instruments.

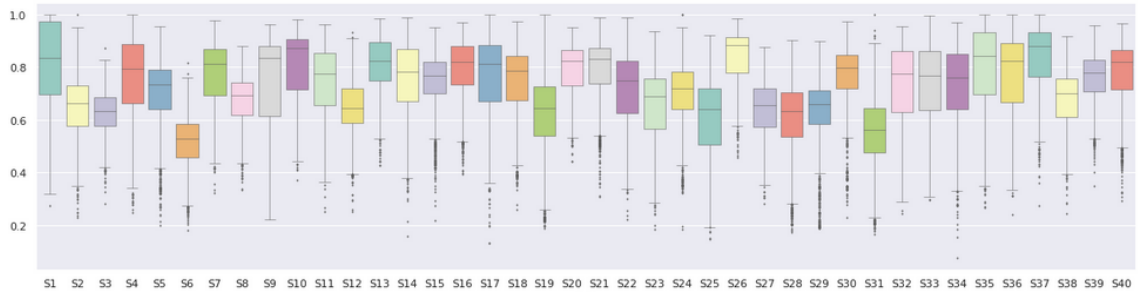

Fig. 22. Piece-wise similarity in terms of notes velocities. Computed on sequences of drums grouped by style (from synthetic dataset). The metric captures well style match, and repetitive groove patterns in terms of velocities are potentially part of the style.

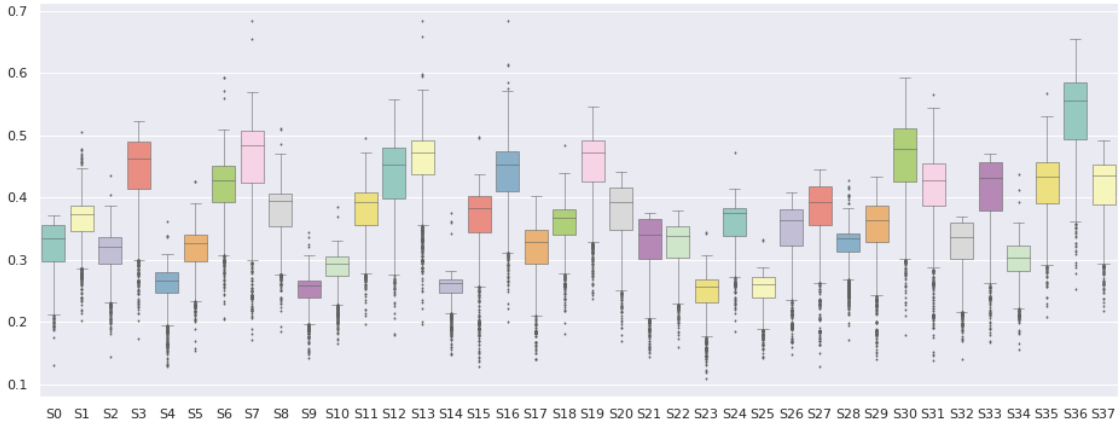

Fig. 23. Piece-wise similarity in terms of notes velocities. Computed on sequences of drums of different styles (opposing styles from synthetic dataset). The metric can differentiate styles as present in synthetic dataset. Different styles do not share similar groove patterns.

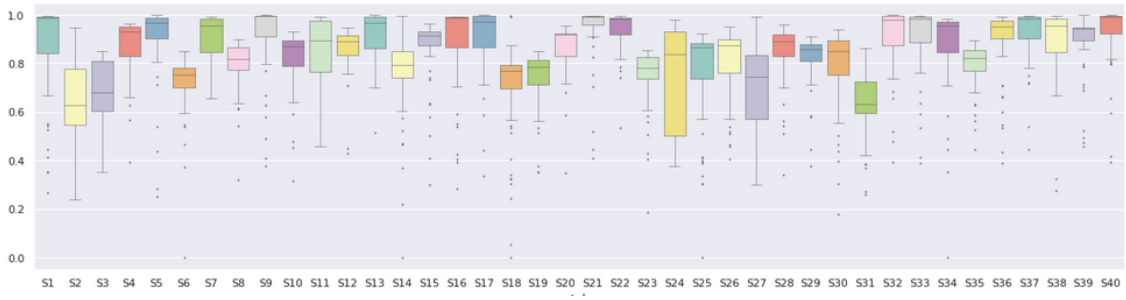

Fig. 24. Piece-wise similarity in terms of notes densities. Computed on sequences of generated drums (using G2G Basic) and their style inputs, grouped by style (from synthetic dataset). The metric captures well style match, and repetitive groove patterns in terms of density are potentially being transferred.

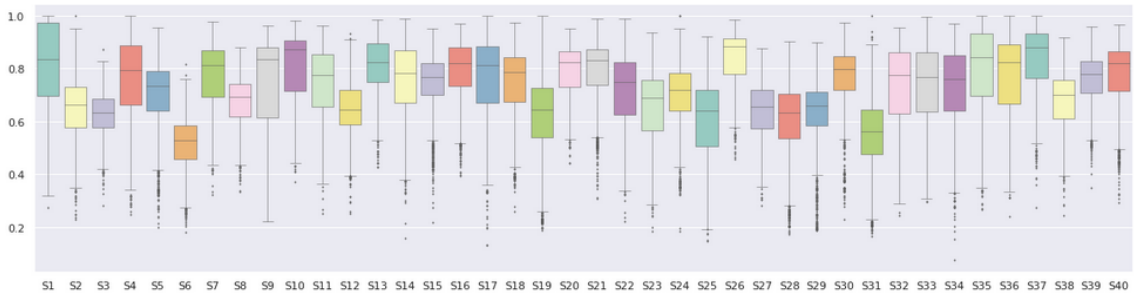

Fig. 25. Piece-wise similarity in terms of notes densities. Computed on sequences of generated drums (using G2G Lakh) and their style inputs, grouped by style (from synthetic dataset). The metric captures well style match, and repetitive groove patterns in terms of density are potentially being transferred.

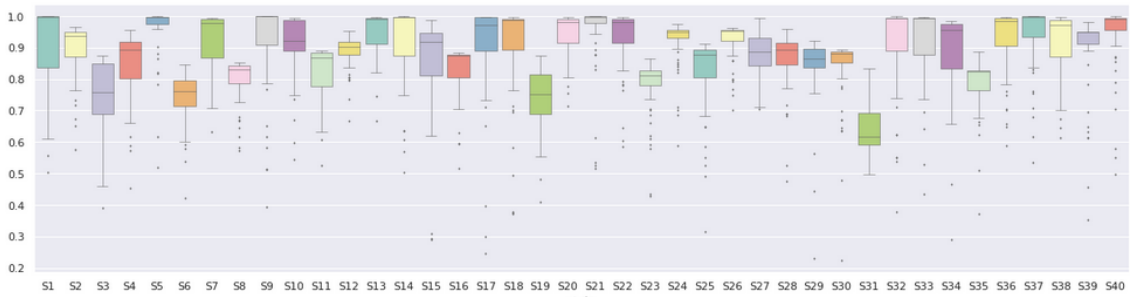

Fig. 26. Piece-wise similarity in terms of notes densities. Computed on sequences of generated drums (using G2G Plus) and their style inputs, grouped by style (from synthetic dataset). The metric captures well style match, and repetitive groove patterns in terms of density are potentially being transferred.

References

Fred Bruford, Olivier Lartillot, SKoT McDonald, and Mark Sandler. Multidimensional similarity modelling of complex drum loops using the groovetoolbox.

Ondřej Cífka, Umut Şimşekli, and Gaël Richard. Groove2groove: One-shot music style transfer with supervision from synthetic data. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28:2638–2650, 2020.

Justin London, Rainer Polak, and Nori Jacoby. Rhythm histograms and musical meter: A corpus study of malian percussion music. *Psychonomic bulletin & review*, 24(2):474–480, 2017.

Shih-Lun Wu and Yi-Hsuan Yang. The jazz transformer on the front line: Exploring the shortcomings of ai-composed music through quantitative measures. *arXiv preprint arXiv:2008.01307*, 2020.